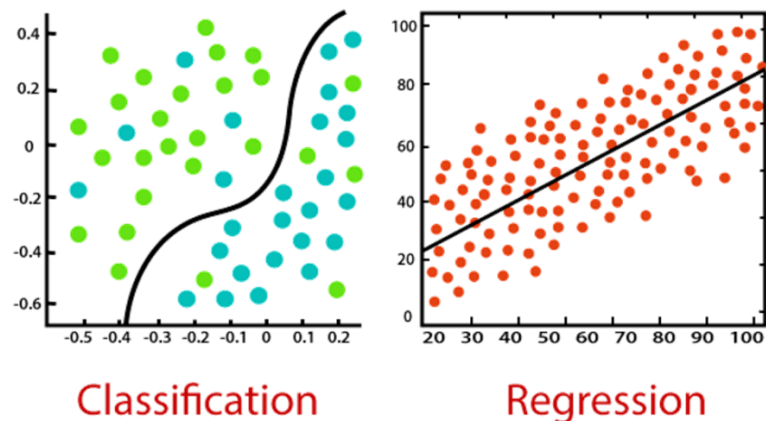


Simon Fraser University**School of Mechatronic Systems Engineering****MSE413 - Machine Learning in Mechatronics****Lab 2 – Classification****Due Date: February 16th, 2025****Background**

Classification is the process of recognizing, understanding, and grouping ideas and objects into preset categories or sub-populations. In supervised machine learning, regression and classification differ in their objectives and outputs. Regression algorithms aim to predict a continuous numerical value, while classification algorithms focus on assigning unseen data into distinct categories. Classification algorithms employ pre-categorized (labeled) training datasets to classify forthcoming (unseen) datasets by predicting the probability that unseen data will fall into one of the predefined categories (classes).



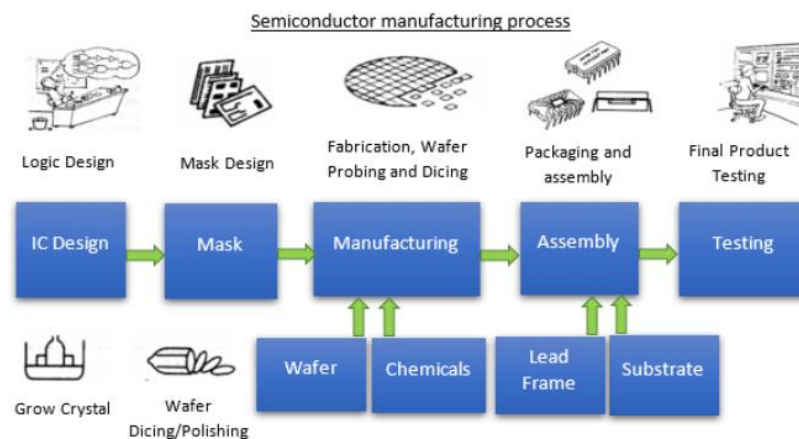
© <https://www.javatpoint.com/regression-vs-classification-in-machine-learning>

There are two major classes of classification problems: Binary-class and Multi-class. In Binary-class classifications, the given dataset is categorized into two categories, whereas in Multi-class classification, the given dataset is categorized into several classes based on the classification rules.

MSE 413- Assignment 2 (Spring 2025)

Binary-class classification

Semiconductor manufacturing process¹



© <https://www.kaggle.com/code/saurabhbagchi/fmst-semiconductor-manufacturing-project/notebook>

A complex, modern semiconductor manufacturing process, as shown in the above figure, is normally under consistent surveillance through the monitoring of signals/features collected from sensors and/or process measurement points. The signals taken from sensors are considered as a feature to predict the efficiency of the final manufactured semiconductor and to identify the most important features/signals contributing to defects in the final product. This will enable an increase in process throughput, a decrease in time to learning, and a reduction in per-unit production costs.

¹ <https://www.kaggle.com/code/saurabhbagchi/fmst-semiconductor-manufacturing-project/notebook>

Each data sample comprises a set of measured features associated with a sample product, and an accompanying label represents a binary pass/fail yield for line testing. A value of -1 denotes “pass” and 1 denotes “fail” and the data time stamp is for that specific test point.

The objective is to build a classifier that predicts the pass/fail result of a semiconductor based on the features/signals measured by the sensors and to identify the most important features/signals contributing to the final product’s efficiency.

Procedure:

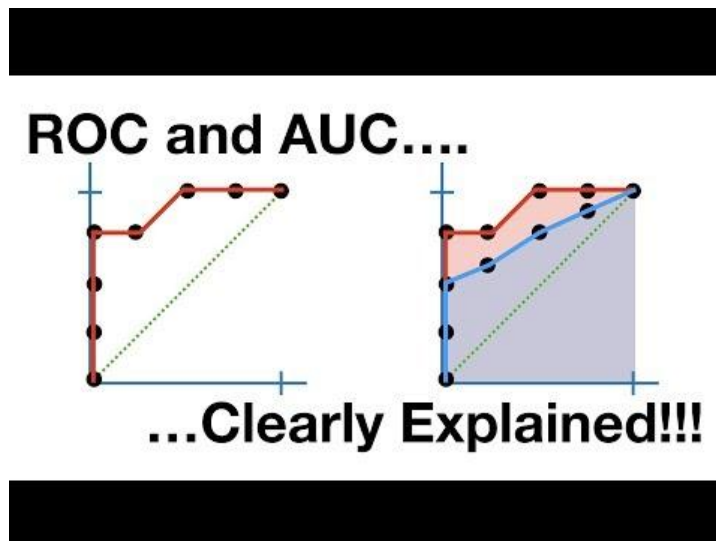
1. Import the “uci-secom.csv” and drop the “Time” column. **(1 point)**
2. Split it comprises 80%) and test set (20 comprises. Consider random effects in splitting the dataset. **(1 point)**
3. Train a random forest classifier on the train set with a number of estimators of 20 and a maximum depth of 20 for the following cases:
 - a) Replacing all the NaN values with 0. **(3 points)**
 - b) Replacing all the NaN values with 0 and drop columns with only a single repeating value. **(3 points)**

You can use the following code to drop columns with a single repeating value:

```
for i in range(0,590):
    First_value=Data.loc[0,[str(i)]]
    count=Data[str(i)].value_counts()[First_value]
    if count[First_value[str(i)]]==len(Data):
        Data= Data.drop([str(i)], axis=1)
```

- c) Replacing all the NaN values with 0, drop columns with only a single repeating value, and drop one of each column pair that is correlated with a factor > 0.8. **(4 points)**
- d) Replacing all the NaN values with the mean value of the feature. **(3 points)**
- e) Replacing all the NaN values with the mean value of the feature and drop columns with only a single repeating value. **(3 points)**

- f) Replacing all the NaN values with the mean value of the feature, drop columns with only a single repeating value, and drop one of each column pair that is correlated with a factor > 0.8 . **(4 points)**
4. Evaluate the performance of the models for cases a to f on the test set by providing the confusion matrix and metrics of classification (accuracy, sensitivity, specificity, and f1 score). **(6 points)**
5. The ROC (Receiver Operating Characteristic) curve visually assesses a binary classification model's performance by illustrating the trade-off between sensitivity and specificity at different thresholds. The Area Under the Curve (AUC) condenses this information into a single value, ranging from 0 to 1, where higher AUC values indicate superior model performance. Watch the following video for more clarification. Plot the ROC curve and print out AUC for cases a to f and discuss the results. **(12 points)**



6. Compare each case's performance and discuss the results. **(4 points)**
7. Gini Importance or Mean Decrease in Impurity (MDI) calculates each feature's importance as the sum of the number of splits (across all trees) that include the feature, proportionally to the number of samples it splits. Using Mean Decrease in Impurity, show the top 20 features that have the highest importance in prediction for cases a to f and discuss the results. **(6 points)**

MSE 413- Assignment 2 (Spring 2024)**Multi-class classification****Classification of Robots from their Conversation**

5 Robots, named 1,2,3,4, and 5 (e.g. 1 means robot 1), are having a uniform conversation, where their conversations have been encoded as 10 numbers at a time in a round-robin fashion. In other words, each robot is generating multi-digit (2 digit, 4 digit, etc,) numbers based on their own internal encoding algorithm to provide a machine level response for their counterpart. The task is to train a model which can predict the robot when given the 10 numbers spoken by it, with good accuracy. The dataset provided is a log of a long conversation among these 5 robots. Table 1 lists a sample of their conversation.

Table 1- a sample of 5 robots' conversation

Robot	num1	num2	num3	num4	num5	num6	num7	num8	num9	num10
1	2	5	2	5	6	2	1	8	1	3
2	1747	1749	1751	1758	1765	1767	1772	1774	1783	1785
3	65056	195168	1561344	7806720	31226880	1.87E+08	7.49E+08	6.75E+09	6.75E+09	6.75E+09
4	2855	2860	2865	2870	2875	2880	2885	2890	2895	2900
5	11440	57200	286000	1430000	7150000	35750000	1.79E+08	8.94E+08	4.47E+09	2.23E+10

1 Getting Started

Load the Robots' Conversation Data Set (Classification of Robots from their conversation sequence.csv) into Google Colab. Show the 10 first rows of the data set. Explain the classification of which robots are more challenging based on the 10 first rows. **(5 points)**

2 Decision tree classifier

- 2.1 Split the dataset into training (80%) and test set (20%). Consider random effects in splitting the dataset. **(1 point)**
- 2.2 Train a decision tree classifier (with default parameters and a depth of 20, and use a normalizer to normalize your features) on the train set. **(2 points)**
- 2.3 Evaluate and discuss the performance of the model on the test set by providing the confusion matrix and metrics of classification (accuracy, sensitivity, specificity, and f1 score). **(3 points)**
- 2.4 Investigate the effect of maximum depth on the accuracy of the model for the following depths: 1, 5, 10, 20, 50, and 100. Discuss the results. **(9 points)**

3 Random forest classifier

- 3.1 Split the dataset into training (80%) and test set (20%). Consider random effects in splitting the dataset. **(1 point)**
- 3.2 Train a random forest classifier on the train set with a number of estimators of 20 and a maximum depth of 20 and use a normalizer to normalize your features. **(2 points)**
- 3.3 Evaluate and discuss the performance of the model on the test set by providing the confusion matrix and metrics of classification (accuracy, sensitivity, specificity, and f1 score). **(3 points)**
- 3.4 Train different random forest classifiers with a number of estimators=[1, 10, 20,100] and maximum depths= [1, 5, 10, 20,100]. **(9 points)**
- 3.5 Compare the accuracy of your trained models. What are the effects of the number of estimators and maximum depth on the accuracy of the model? **(5 points)**

- 3.6 Using Mean Decrease in Impurity, show which feature has the highest importance in prediction. **(5 points)**

4 K-Nearest Neighbors

- 4.1 Split the dataset into training (80%) and test set (20%). Consider random effects in splitting the dataset. **(1 point)**
- 4.2 Train a KNN classifier on the train set with $K=1$ and use a normalizer to normalize your features. **(2 points)**
- 4.3 Evaluate and discuss the performance of the model on the test set by providing the confusion matrix and metrics of classification (accuracy, sensitivity, specificity, and f1 score). **(3 points)**
- 4.4 Train ten different KNN classifiers with $K=[1, 2, 3, \dots, 9, 10]$. **(9 points)**
- 4.5 Compare the accuracy of your trained models by plotting accuracy versus K (horizontal axis: K , vertical axis: accuracy). Which K resulted in the highest accuracy? Discuss the results. **(5 points)**

5 Support vector classifier

- 5.1 Split the dataset into training (80%) and test set (20%). Consider random effects in splitting the dataset. **(1 point)**
- 5.2 Train a support vector classifier on the train set where $C=10000$ and kernel is “rbf” and use a normalizer to normalize your features. Were you able to train the model? **(2 points)**
- 5.3 Generally, SVM is not able to handle a large number of data. Therefore, one way is to divide the data set into several data chunks and train several sub-SVMs. We name the algorithm as Map-Reduce. For instance, the present dataset has about 500,000 instances. Thus, the train set has 400,000 instances. In the map phase, we divide the train set into 20 data chunks with 20,000 instances. As a result, we can train 20 SVM models where each SVM is trained on only one data chunk. In the reduce phase, we predict the test set using 20 SVM models. The final answer is obtained using the majority of voting of all SVM models. You may read the following papers to better understand the concept. Use the below pseudo

code to deal with the current big data. You need to fill "?" in the code. Explain how the below pseudo code works. **(12 points)**

Ashouri, Mahyar, et al. "A big data-handling machine learning model for membrane-based absorber reactors in sorption heat transformers." *Energy Conversion and Management* 292 (2023): 117376.

Khatibi, Toktam, Naghme Kheyrikoochaksarayee, and Mohammad Mehdi Sepehri. "Analysis of big data for prediction of provider-initiated preterm birth and spontaneous premature deliveries and ranking the predictive features." *Archives of gynecology and obstetrics* 300 (2019): 1565-1582.

Pseudo code:

```
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import Normalizer
import pickle
import statistics
import pandas as pd
import math

Data = pd.read_csv(Path("/content/Classification of Robots from their
conversation sequence.csv"))
Data= Data.dropna()

train_set_Map, test_set= train_test_split(Data, test_size=0.2,
random_state=42)

Data_Map = train_set_Map.copy()
Data_test_R= test_set.drop(["Robot"], axis=1)
Data_labels_test_R= test_set["Robot"].copy()

chunk_number=10
chunk_size =math.ceil(len(?) /chunk_number)
data_chunked={}
m=0

for i in range(0, len(?), chunk_size):
    if m==chunk_number-1:
        chunk_size= len(Data_Map)- m *chunk_size
        data_chunked[str("chunk"+str(m))] = Data_Map[i:i+?]
    print(chunk_size)
```

```

m=m+1

for i in range(0, chunk_number):
    Data_M = data_chunked[str("chunk"+str(i))].drop(["?"], axis=1)
    Data_labels_M = data_chunked[str("chunk"+str(i))]["?"].copy()
    SVMC = Pipeline(steps=[('Normal', Normalizer()), ('SVMC',
SVC(C=10000000, kernel='rbf'))])
    SVMC.fit(?,?)
    pickle.?(SVMC, open('SVMC.pkstr(i)', 'wb'))
    print(i)

OUT_SVMC={}
store = [[0 for col in range(chunk_number)] for row in
range(len(Data_test_R))]
Lables_SVMC_predicted=[[0 for col in range(1)] for row in
range(len(Data_test_R))]

for i in range(0,?):
    SVMC1 = pickle.?(open('SVMC.pkstr(i)', 'rb'))
    OUT_SVMC[str("chunk"+str(i))]=SVMC1.predict(?)
    print(i)

for j in range(0, len(Data_test_R)):
    for i in range(0, chunk_number):
        store[j][i]=OUT_SVMC[str("chunk"+str(i))][j]
    Lables_SVMC_predicted[j]=statistics.?(store[j])

```

- 5.4 Train 3 models for 10, 20, and 40 data chunks (where C= 1000000 and kernel is rbf) and predict the test set. **(5 points)**
- 5.5 Evaluate and discuss the performance of each model by providing the confusion matrix, metrics of classification (accuracy, sensitivity, specificity, and f1 score), and computation time. (10 points) **(3 points)**
- 5.6 Compare the accuracy and computational time of your trained models and discuss the results. **(3 points)**
- 5.7 Evaluate the effect of margin on the accuracy for different values of C=[1,10,100,1000, 10000,100000, and 1000000] and discuss the results (chunk number= 20). **(4 points)**

6 Comparison between the mentioned models

Compare the best performance of each classifier and discuss which classifier is the most accurate one and why. **(5 points)**

Submitting your Assignment

The assignment must be submitted online to Canvas with the following structure:

For each group, you must submit:

1. **A report**- The assignment report must be in **PDF format**, called **report.pdf**. This report must contain all the figures and discussions/explanations for the questions.
2. **A link to access the code in Google Colab.**